

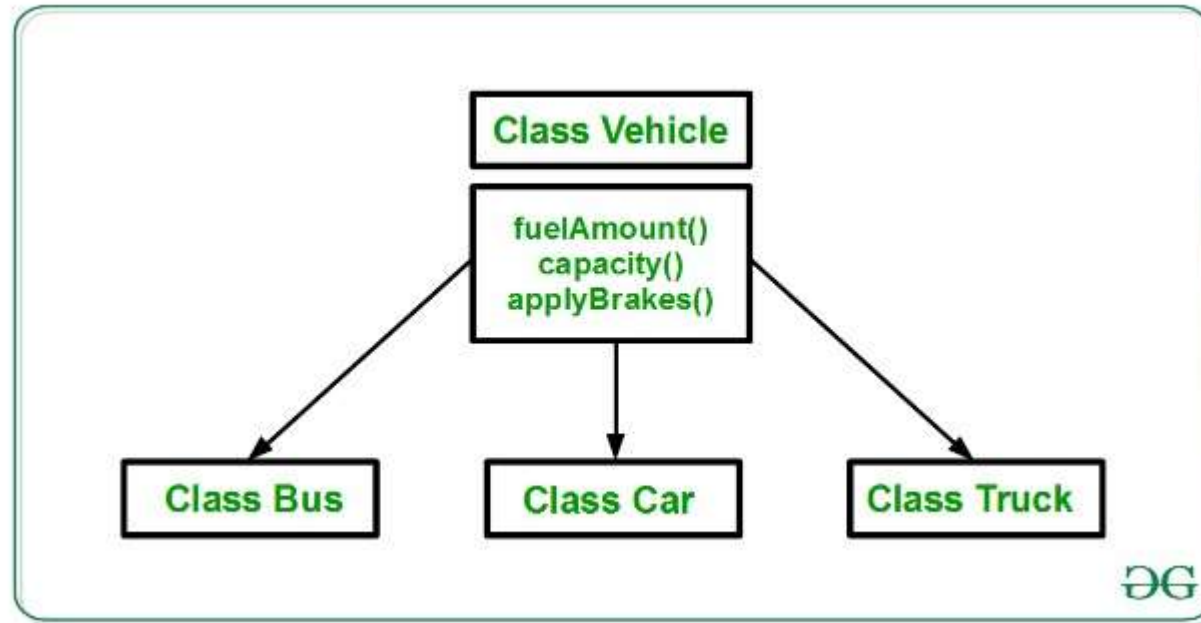
Chapter Three

Inheritance and Polymorphism

➤ Inheritance

- **Inheritance** is one of the core concepts of Object-Oriented Programming (OOP).
- Allows you to *drive a new class* from *an existing one*.
- It is used to create more specialized child classes (i.e., subclasses) from a parent class (i.e., a superclass)
 - ✓ E.g. driving *Bus* from *Vehicle* class
 - ✓ Sub class acquires all the *properties* and *behaviors* of the super class, and also can add its own unique properties and methods
- Inheritance represents the **IS-A relationship**, also known as parent-child relationship
 - IS-A relationship denotes “*one object is type of another*”.
 - ✓ E.g. *Bus* IS-A *Vehicle*.

Cont....



■ Why use inheritance in java ?

- ✓ *Improves code reusability / Reduces redundancy*
- ✓ *Helps to achieve **runtime polymorphism** – through method overriding*
- ✓ *Good Hierarchy Representation: Organize related classes into parent-child relationships.*

Cont....

- In Java, Inheritance can be implemented with *extends* (in case of class) and *implements* (in case of interface) keywords.

❖ *Extends*: Syntax of Java Inheritance

```
class ParentClass
{
    //fields and methods
}

class ChildClass extends ParentClass
{
    //attributes and methods
}
```

- ✓ The `extends` keyword indicates that you are making a new class that derives from an existing class.
- ✓ Child class inherits all *non-private members* of the parent class.

Cont....

❖ Example

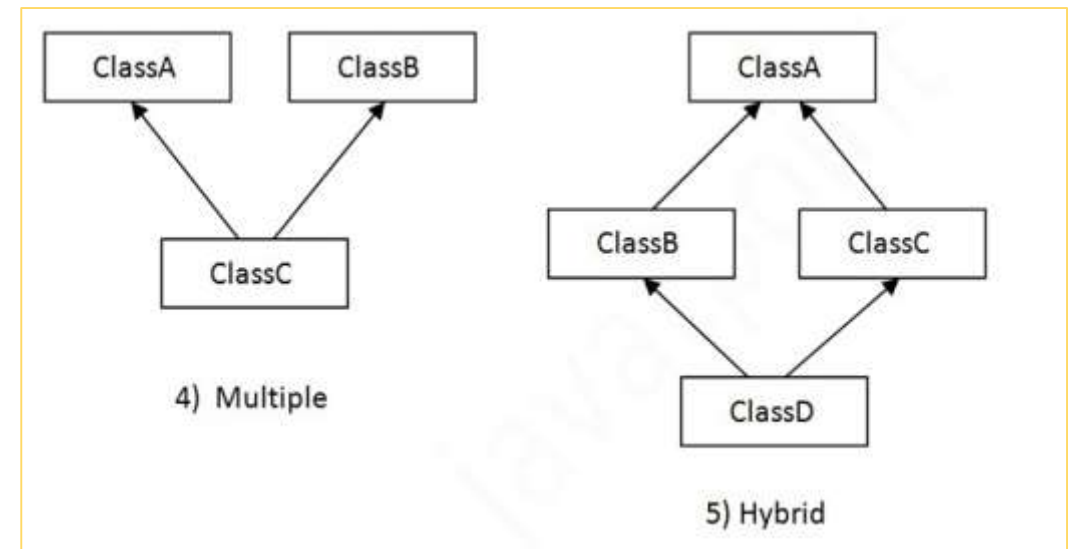
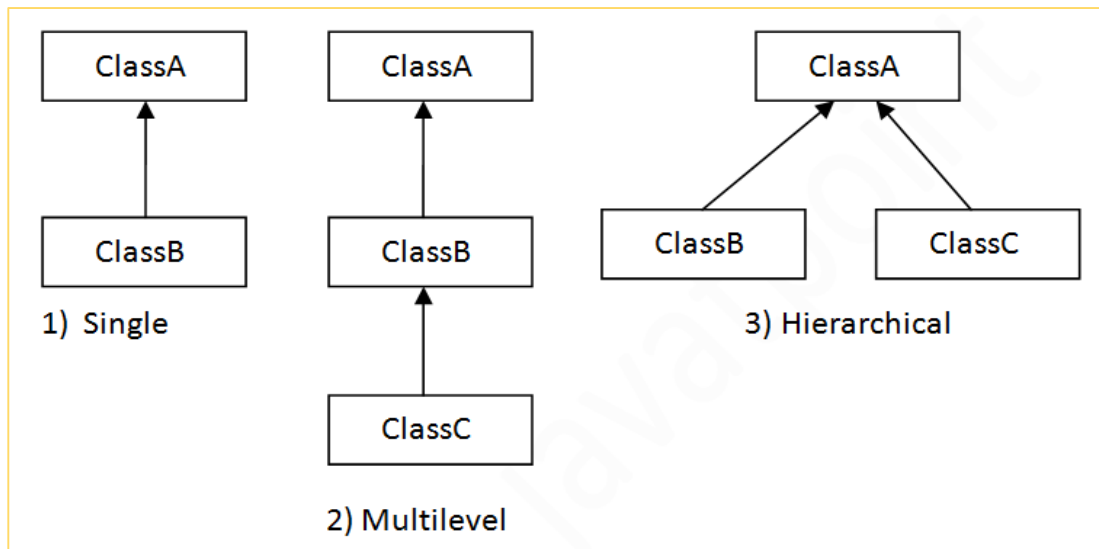
```
class Vehicle
{
    void run() {
        System.out.println("This vehicle is running.");
    }
}
class Bus extends Vehicle
{
    void carryPassengers()
    {
        System.out.println("The bus carries passengers.");
    }
}

public class HELLO {
    public static void main(String[] args)
    {
        Bus b = new Bus();
        b.run();           // Inherited method
        b.carryPassengers(); // Own method
    }
}
```

```
run:
This vehicle is running.
The bus carries passengers.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Types of inheritance in java

- On the basis of class, there can be *three types of inheritance* in java: *single*, *multilevel* and *hierarchical*.
 - To avoid ambiguity Java does NOT support *multiple* and *hybrid* inheritance with classes directly. They are both supported through *interface* only



Cont....

A) Single Inheritance

- A class inherits from only one parent class.
- *Benefit:* Clear and easy to maintain.

```
class Vehicle{
    void start() { System.out.println("Vehicle starting"); }
}

class Bus extends Vehicle
{
    void stop() { System.out.println("Bus stopping"); }
}

public class HELLO {
    public static void main(String[] args) {
        Bus bus = new Bus();
        bus.start(); // Inherited from Vehicle
        bus.stop();  // Defined in Bus
    }
}
```

Vehicle starting
Bus stopping

B) Multi level Inheritance

- A class is derived from a class which is also derived from another class.

```
class Vehicle {  
    void start() { System.out.println("Vehicle starting"); }  
}  
  
class Bus extends Vehicle  
{  
    void stop() { System.out.println("Bus stopping"); }  
}  
  
class SchoolBus extends Bus  
{  
    void pickUp() { System.out.println("Picking up students"); }  
}  
  
public class HELLO  
{  
    public static void main(String[] args) {  
        SchoolBus sb = new SchoolBus();  
        sb.start(); // Inherited from Vehicle  
        sb.stop();  // Inherited from Bus  
        sb.pickUp(); // Defined in SchoolBus  
    }  
}
```

Vehicle starting
Bus stopping
Picking up students

C) Hierarchical Inheritance

- Multiple classes inherit from a single parent class.

```
class Vehicle {
    void start() { System.out.println("Vehicle starting"); }
}
class Bus extends Vehicle
{
    void stop() { System.out.println("Bus stopping"); }
}
class Truck extends Vehicle
{
    void load() { System.out.println("Truck loading"); }
}
public HELLO {
    public static void main(String[] args) {
        Bus bus = new Bus();
        bus.start(); // Inherited from Vehicle
        bus.stop();  // Defined in Bus

        Truck truck = new Truck();
        truck.start(); // Inherited from Vehicle
        truck.load();  // Defined in Truck
    }
}
```

```
Vehicle starting
Bus stopping
Vehicle starting
Truck loading
```

Cont....

❖ Java Object Class

- The Object class is *the root (superclass) of all classes* in Java.
- *Every class in Java implicitly extends the Object class* (unless it explicitly extends another class).
- It is defined in *java.Lang.Object*
- If no inheritance is specified when a class is defined, the superclass of the class is Object by default.
- For example, the following two class definitions are the same:

```
public class  ClassName {  
    ...  
}
```

Equivalent

```
public class  ClassName extends Object {  
    ...  
}
```

Cont....

- Common Methods of the **Object** Class:

Method Name	Description
toString()	Returns a string representation of the object.
equals(Object o)	Compares two objects for equality.
hashCode()	Returns a hash code for the object.
getClass()	Returns the runtime class of the object.
clone()	Creates and returns a copy of the object.
finalize()	Called before garbage collection.
wait(), notify(), notifyAll()	Used for thread synchronization.

- You can override some Object methods in your class (like toString(), equals(), etc.).

Cont....

- Example:

```
class Vehicle
{
    int speed = 50;
}

public class HELLO
{
    public static void main(String[] args)
    {
        Vehicle v = new Vehicle();
        System.out.println(v.toString()); // from Object class
    }
}
```

- Prints something like *Vehicle*@15db9742, which is the string representation of the object
- That is the default implementation of `toString()` method in object class. But you can change what it does using method overriding

➤ Constructor in Inheritance

- The constructor for the superclass *constructs the superclass portion of the object*, and the *constructor for the subclass constructs the subclass* part.
- The child class constructor is responsible for calling the parent class constructor.
- In Java `super()` keyword is used *to call the parent class constructor*, and *access parent class methods* or *variables*.
 - Syntax: `super()`, or `super(parameters)`;
 - `super()` invokes the no-argument constructor of its superclass, and
 - If the parent class has a constructor with parameters, the child class must explicitly call it using `super(arguments)` having parameters.
 - If no `super()` is written in the child class constructor, and parent class has no parameterized constructor, Java by default calls the parent's no-argument constructor automatically.

Cont....

- Example:

```
class Vehicle {
    Vehicle(String type)           // Parent constructor
    {
        System.out.println("Vehicle type: " + type);
    }
}

class Bus extends Vehicle
{
    Bus(){                          // Child constructor
        super("City Bus");         // Explicitly calling the parent constructor
        System.out.println("Bus constructor");
    }
}

public class HELLO
{
    public static void main(String[] args)
    {
        Bus b = new Bus();
    }
}
```

```
run:
Vehicle type: City Bus
Bus constructor
BUILD SUCCESSFUL (total time: 0 seconds)
```

Cont....

- In Java, `super()` must be called within the child class constructor, and specifically *as the first statement* in the constructor.
- You cannot call `super()` *outside of the constructor* itself or *after other statements* in the child constructor.

➤ Using super to Access Superclass Members

- ✓ If both parent & child class have the same variable or method name
- This usage has the following general form:
`super.member`
- Here, member can be either a method or an instance variable.

Example: class **Vehicle** {

```
    String type = "General Vehicle";
```

```
    void displayInfo() {
```

```
        System.out.println("This is a vehicle.");
```

```
    }
```

```
}
```

```
class Bus extends Vehicle {
```

```
    String type = "City Bus";    // The same variable name in parent and child class
```

```
    void showDetails() {
```

```
        System.out.println("Child type: " + type);                                //City Bus
```

```
        System.out.println("Parent type: " + super.type);    // General Vehicle
```

```
        super.displayInfo();                                // Call parent class method
```

```
    }
```

```
}
```

```
public class HELLO {
```

```
    public static void main(String[] args) {
```

```
        Bus myBus = new Bus();
```

```
        myBus.showDetails();
```

```
    } }
```

Child type: City Bus

Parent type: General Vehicle

This is a vehicle.

BUILD SUCCESSFUL (total time: 0 seconds)

Cont....

➤ Method Overriding

- Is a feature that *allows a subclass to provide a specific implementation of a method that is already defined in its superclass.*
 - Child class can override parent class methods.
- Difference between overloading and overriding
 - *Overloading* - means to define multiple methods *with the same name but different signatures.*
 - *Overriding* - means to *provide a new implementation for a method in the subclass*
- To override the method in the child class must have the *same name, return type, and parameters* as the one in the parent class.

Cont....

- Example 1: class *Vehicle*

```
{
    void run(){
        System.out.println("The vehicle is running...");
    }
}
class Bus extends Vehicle
{
    @Override
    void run(){
        System.out.println("The bus is running");
    }
}
public class HELLO
{
    public static void main(String[] args)
    {
        Vehicle v = new Bus();
        v.run();           // Calls overridden method in Bus
    }
}
```

Cont....

- Only inherited methods can be overridden (*not private methods*).
- It is used to achieve runtime polymorphism.
- Constructors cannot be overridden.
- *Private*, *static*, and *final* methods cannot be overridden.
- Use the *@Override* annotation to make overriding explicit (*optional but recommended*).

Cont....

- Example 2: Lets override a method defined inside object class (*toString()* method), used as example in slide 12

```
class Vehicle
{
    int speed = 50;

    @Override
    public String toString()
    {
        return "The method is overridden";
    }
}

public class HELLO
{
    public static void main(String[] args)
    {
        Vehicle v = new Vehicle();
        System.out.println(v.toString()); // from Object class
    }
}
```

```
run:
The method is overridden
BUILD SUCCESSFUL (total time: 0 seconds)
```

Cont....

▪ *Final Keyword & Inheritance*

-  if you use final keyword with the class it means *the class cannot be inherited* (extended) by any other class.
- Prevents further modification of the class through inheritance.
 - ✓ And because of this you cannot override any of its methods
- But, you can still create objects of a final class.
- Example:

```
final class Vehicle{ ..... }  
class Bus extends Vehicle { ....} // Error
```

Cont....

- *Member Access and Inheritance*

- Even though a subclass includes all of the members of its superclass, it *cannot access those members of the superclass that have been declared private*.
 - *private*: Not inherited.
 - *default (no modifier)*: Inherited within the same package.
 - *protected*: Inherited within same package + subclasses outside package.
 - *public*: Always inherited.

Cont....

Feature

final

private

Purpose

Prevents **modification/extension**

Restricts **visibility/access**

Applied to class

Prevents inheritance

✗ Can't be used on top-level classes

Applied to method

Prevents overriding

Accessible only within the same class

Applied to variable

Makes it constant (unchangeable)

Accessible only within the same class

Inheritance

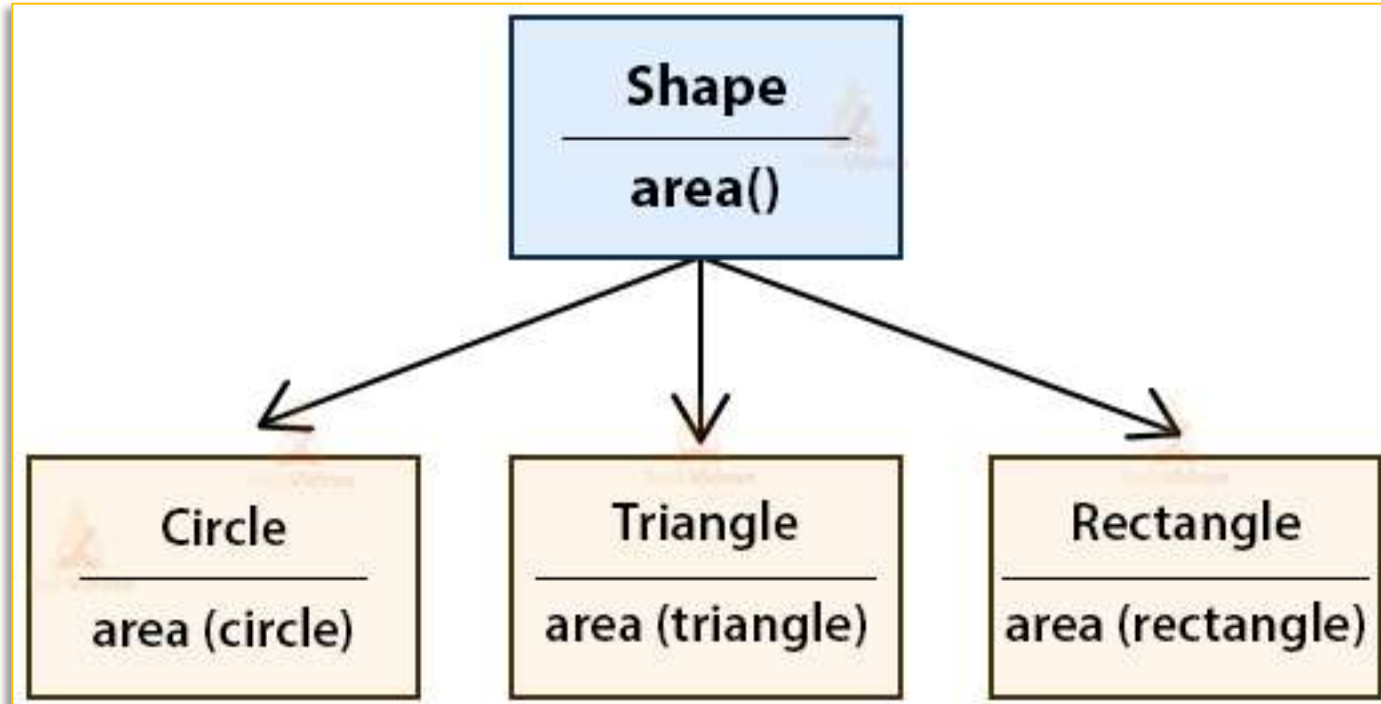
final class = ✗ no subclass allowed

private methods/fields are just **hidden**, but class can still be extended

➤ Polymorphism

- Polymorphism is a core concept in Object-Oriented Programming (OOP).
- The term polymorphism comes from the Greek words "*poly*" (many) and "*morph*" (form),
 - ✓ So polymorphism means *many forms* (*one thing that have many behavior*).
- It allows one interface (like a method or object) to be used in different ways depending on the context.
- Importance
 - ✓ To write *flexible* and *reusable* code.
 - ✓ To *enable one name to behave differently* based on object type or method signature.
 - ✓ To *simplify code maintenance* and extension.

Cont....



Cont....

🔑 Two Types of Polymorphism in Java

1. Compile-Time Polymorphism (Static Polymorphism):

- *Achieved by method overloading* and constructor overloading.
- The *decision of which method or constructor to call is made at compile time*.

2. Runtime Polymorphism (Dynamic Polymorphism):

- *Achieved by method overriding*
- The *decision of which method to call is made at runtime* based on the object type.

Cont....

- *Compile-Time Polymorphism (Method Overloading)*
 - Method Overloading occurs when *multiple methods have the same name but differ in parameters* (number or type)
 - The method to be called is **determined at compile time**.

```
Example 1: class AreaCalculator {  
    void calculateArea(double radius) {  
        System.out.println("Circle Area: " + (Math.PI * radius * radius));  
    }  
    void calculateArea(double length, double width) {  
        System.out.println("Rectangle Area: " + (length * width));  
    }  
    void calculateArea(double base, double height, boolean isTriangle) {  
        System.out.println("Triangle Area: " + (0.5 * base * height));  
    }  
}  
  
public class HELLO {  
    public static void main(String[] args) {  
        AreaCalculator calc = new AreaCalculator();  
        calc.calculateArea(5);  
        calc.calculateArea(4, 6);  
        calc.calculateArea(3, 7, true);  
    }  
}
```

```
Circle Area: 78.53981633974483  
Rectangle Area: 24.0  
Triangle Area: 10.5  
BUILD SUCCESSFUL (total time: 0 seconds)
```

- Example 2: class *Vehicle*

```
{
    void run(){
        System.out.println("The vehicle is running...");
    }
    void run(String type)
    {
        System.out.println("The " + type + " is running...");
    }
    void run(int speed)
    {
        System.out.println("The vehicle is running at " + speed + " km/h");
    }
}

public class HELLO {
    public static void main(String[] args)
    {
        Vehicle v = new Vehicle();
        v.run();           // Calls run() method
        v.run("car");      // Calls run(String) method
        v.run(60);         // Calls run(int) method
    }
}
```

The vehicle is running...

The car is running...

The vehicle is running at 60 km/h

Cont....

- *Runtime Polymorphism (Method Overriding)*
 - Method Overriding occurs *when a subclass provides a specific implementation of a method that is already defined in its superclass.*
 - The method to be executed is **determined at runtime** based on the actual object type, not the reference type.

```
Example 1: class Shape {  
    void calculateArea() { System.out.println("Area not defined"); }  
}  
  
class Circle extends Shape {  
    void calculateArea() {  
        System.out.println("Circle Area: " + (Math.PI * 5 * 5));  
    }  
}  
  
class Rectangle extends Shape {  
    void calculateArea() {  
        System.out.println("Rectangle Area: " + (4 * 6));  
    }  
}  
  
class Triangle extends Shape {  
    void calculateArea() {  
        System.out.println("Triangle Area: " + (0.5 * 3 * 7));  
    }  
}
```

Cont....

```
_____public class HELLO
{
    public static void main(String[] args)
    {
        Circle s1 = new Circle();
        Rectangle s2 = new Rectangle();
        Triangle s3 = new Triangle();

        s1.calculateArea();
        s2.calculateArea();
        s3.calculateArea();
    }
}
```

```
Circle Area: 78.53981633974483
Rectangle Area: 24.0
Triangle Area: 10.5
BUILD SUCCESSFUL (total time: 0 seconds)
```


Cont....

- Example 2: class *Vehicle*

```
{
    void run(){
        System.out.println("The vehicle is running...");
    }
}
class Bus extends Vehicle
{
    @Override
    void run(){
        System.out.println("The bus is running");
    }
}
public class HELLO
{
    public static void main(String[] args)
    {
        Vehicle v = new Bus();
        v.run();    // Calls overridden method in Bus class
    }
}
```

run:

The bus is running

BUILD SUCCESSFUL (total time: 0 seconds)

Cont....

- Abstract Class and Overriding

- An abstract class is *a class that cannot be instantiated*.
- It may contain
 - *abstract methods* (without a body) and
 - *concrete methods* (with a body).
- You can't create an object of an abstract class.
- Can contain *constructors, fields, and non-abstract methods*.
- Used when *we want to force subclasses to implement certain methods*.

Example:

```
abstract class Vehicle {  
    abstract void start(); // abstract method  
    void fuel()           // concrete method  
    {  
        System.out.println("Vehicle needs fuel");  
    }  
}  
  
class Bus extends Vehicle {  
    @Override  
    void start()  
    {  
        System.out.println("Bus is starting...");  
    }  
}  
  
public class HELLO {  
    public static void main(String[] args)  
    {  
        Vehicle v = new Bus();  
        v.start();           // Calls overridden method  
        v.fuel();            // Calls concrete method from abstract class  
    }  
}
```

```
run:  
Bus is starting...  
Vehicle needs fuel  
BUILD SUCCESSFUL (total time: 0 seconds)
```

The END !